

50 Android Apps with Jetpack Compose

Complete Kotlin Code Examples for Learning

Learn Android development with 50 complete, working apps using Jetpack Compose. Each app includes full source code you can copy and run. From simple Hello World to complex apps with state management, animations, and API integration. Perfect for beginners and intermediate developers.

SIMPLE APPS (1-15)

App 1: Hello World App

Display a simple greeting message

```
@Composable fun HelloWorldApp() { Box( modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center ) { Text( text = "Hello, Jetpack Compose!", fontSize = 24.sp, fontWeight = FontWeight.Bold, color = Color.Blue ) } }
```

App 2: Counter App

Simple counter with increment/decrement buttons

```
@Composable fun CounterApp() { var count by remember { mutableStateOf(0) } Column( modifier = Modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center ) { Text( text = "Count: $count", fontSize = 48.sp, fontWeight = FontWeight.Bold ) Spacer(modifier = Modifier.height(16.dp)) Row { Button(onClick = { count-- }) { Text("-", fontSize = 24.sp) } Spacer(modifier = Modifier.width(16.dp)) Button(onClick = { count++ }) { Text("+", fontSize = 24.sp) } } Spacer(modifier = Modifier.height(16.dp)) Button(onClick = { count = 0 }) { Text("Reset") } } }
```

App 3: Text Input App

Display entered text in real-time

```
@Composable fun TextInputApp() { var text by remember { mutableStateOf("") } Column( modifier = Modifier .fillMaxSize() .padding(16.dp), verticalArrangement = Arrangement.Center ) { OutlinedTextField( value = text, onValueChange = { text = it }, label = { Text("Enter your name") }, modifier = Modifier.fillMaxWidth() ) Spacer(modifier = Modifier.height(16.dp)) if (text.isNotEmpty()) { Text( text = "Hello, $text!", fontSize = 20.sp, fontWeight = FontWeight.Bold ) } } }
```

App 4: Toggle Switch App

Toggle between light and dark mode

```
@Composable fun ToggleSwitchApp() { var isDarkMode by remember { mutableStateOf(false) } val backgroundColor = if (isDarkMode) Color.DarkGray else Color.White val textColor = if (isDarkMode) Color.White else Color.Black Box( modifier = Modifier.fillMaxSize() .background(backgroundColor), contentAlignment = Alignment.Center ) { Column(horizontalAlignment = Alignment.CenterHorizontally) { Text( text = if (isDarkMode) "Dark Mode" else "Light Mode", fontSize = 24.sp, color = textColor ) Spacer(modifier = Modifier.height(16.dp)) Switch( checked = isDarkMode, onCheckedChange = { isDarkMode = it } ) } } }
```

App 5: Simple List App

Display a scrollable list of items

```
@Composable fun SimpleListApp() { val items = listOf( "Apple", "Banana", "Cherry", "Date", "Elderberry", "Fig", "Grape", "Honeydew", "Kiwi", "Lemon" ) LazyColumn( modifier = Modifier.fillMaxSize(), contentPadding = PaddingValues(16.dp) ) { items(items) { item -> Card( modifier = Modifier .fillMaxWidth() .padding(vertical = 4.dp), elevation = CardDefaults.cardElevation(4.dp) ) { Text( text = item, modifier = Modifier.padding(16.dp), fontSize = 18.sp ) } } } }
```

App 6: Color Picker App

Change background color with buttons

```
@Composable fun ColorPickerApp() { var backgroundColor by remember { mutableStateOf(Color.White) } Column( modifier = Modifier .fillMaxSize() .background(backgroundColor), horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center ) { Text("Choose a color:", fontSize = 20.sp) Spacer(modifier = Modifier.height(16.dp)) Row { ColorButton(Color.Red) { backgroundColor = Color.Red } ColorButton(Color.Green) { backgroundColor = Color.Green } ColorButton(Color.Blue) { backgroundColor = Color.Blue } } Row { ColorButton(Color.Yellow) { backgroundColor = Color.Yellow } ColorButton(Color.Cyan) { backgroundColor = Color.Cyan } ColorButton(Color.Magenta) { backgroundColor = Color.Magenta } } } @Composable fun ColorButton(color: Color, onClick: () -> Unit) { Button( onClick = onClick, colors = ButtonDefaults.buttonColors(containerColor = color), modifier = Modifier .padding(4.dp) .size(60.dp) ) { } }
```

App 7: Image Display App

Display an image with description

```
@Composable fun ImageDisplayApp() { Column( modifier = Modifier .fillMaxSize() .padding(16.dp), horizontalAlignment = Alignment.CenterHorizontally ) { Text( text = "Image Viewer", fontSize = 24.sp, fontWeight = FontWeight.Bold ) Spacer(modifier = Modifier.height(16.dp)) Image( painter = painterResource(id = R.drawable.sample_image), contentDescription = "Sample Image", modifier = Modifier .size(200.dp) .clip(RoundedCornerShape(16.dp)), contentScale = ContentScale.Crop ) Spacer(modifier = Modifier.height(16.dp)) Text( text = "Beautiful landscape photo", fontSize = 16.sp, textAlign = TextAlign.Center ) } }
```

App 8: Checkbox List App

Todo list with checkboxes

```
@Composable fun CheckboxListApp() { var tasks by remember { mutableStateOf( listOf( "Buy groceries" to false, "Walk the dog" to false, "Read a book" to false, "Exercise" to false ) ) } LazyColumn( modifier = Modifier .fillMaxSize() .padding(16.dp) ) { item { Text( text = "My Tasks", fontSize = 24.sp, fontWeight = FontWeight.Bold ) Spacer(modifier = Modifier.height(16.dp)) } itemsIndexed(tasks) { index, (task, checked) -> Row( modifier = Modifier .fillMaxWidth() .padding(vertical = 4.dp), verticalAlignment = Alignment.CenterVertically ) { Checkbox( checked = checked, onCheckedChange = { isChecked -> tasks.toMutableList().also { it[index] = task to isChecked } } ) Text( text = task, fontSize = 16.sp, textDecoration = if (checked) TextDecoration.LineThrough else null ) } } }
```

App 9: Timer App

Simple countdown timer

```
@Composable fun TimerApp() { var seconds by remember { mutableStateOf(0) } var isRunning by remember { mutableStateOf(false) } LaunchedEffect(isRunning) { while (isRunning && seconds > 0) { delay(1000) seconds-- } if (seconds == 0) isRunning = false } Column( modifier = Modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center ) { Text( text = "%02d:%02d".format(seconds / 60, seconds % 60), fontSize = 48.sp, fontWeight = FontWeight.Bold ) Spacer(modifier = Modifier.height(16.dp)) Row { Button(onClick = { seconds += 60 }, enabled = !isRunning) { Text("+1 Min") } Spacer(modifier = Modifier.width(8.dp)) OutlinedTextField( value = fahrenheit, onValueChange = { fahrenheit = !isRunning }, enabled = seconds > 0 ) { Text(if (isRunning) "Pause" else "Start") } Spacer(modifier = Modifier.width(8.dp)) Button(onClick = { seconds = 0; isRunning = false }) { Text("Reset") } } } }
```

App 10: Temperature Converter

Convert between Celsius and Fahrenheit

```
@Composable fun TemperatureConverter() { var celsius by remember { mutableStateOf("") } var fahrenheit by remember { mutableStateOf("") } Column( modifier = Modifier .fillMaxSize() .padding(16.dp), verticalArrangement = Arrangement.Center ) { Text("Temperature Converter", fontSize = 24.sp, fontWeight = FontWeight.Bold) Spacer(modifier = Modifier.height(16.dp)) OutlinedTextField( value = celsius, onValueChange = { celsius = it fahrenheit = it.toDoubleOrNull()?.let { c -> (c * 9/5 + 32).toString() } ?: "" }, label = { Text("Celsius") }, modifier = Modifier.fillMaxWidth() ) Spacer(modifier = Modifier.height(8.dp)) OutlinedTextField( value = max, onValueChange = { max = it }, label = { Text("Max") }, modifier = it celsius = it.toDoubleOrNull()?.let { f -> ((f - 32) * 5/9).toString() } ?: "" }, label = { Text("Fahrenheit") }, modifier = Modifier.fillMaxWidth() ) } }
```

App 11: Random Number Generator

Generate random numbers within range

```
@Composable fun RandomNumberApp() { var randomNumber by remember { mutableStateOf(0) } var min by remember { mutableStateOf("1") } var max by remember { mutableStateOf("100") } Column( modifier = Modifier .fillMaxSize() .padding(16.dp), horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center ) { Text( text = "Random Number", fontSize = 24.sp, fontWeight = FontWeight.Bold ) Spacer(modifier = Modifier.height(32.dp)) Text( text = randomNumber.toString(), fontSize = 72.sp, fontWeight = FontWeight.Bold, color = Color.Blue ) Spacer(modifier = Modifier.height(32.dp)) Row { OutlinedTextField( value = min, onValueChange = { min = it }, label = { Text("Min") }, modifier = Modifier.weight(1f) ) Spacer(modifier = Modifier.width(8.dp)) OutlinedTextField( value = max, onValueChange = { max = it }, label = { Text("Max") }, modifier = Modifier.weight(1f) ) } Spacer(modifier = Modifier.height(16.dp)) Button(onClick = { val minVal = min.toIntOrNull() ?: 1 val maxVal = max.toIntOrNull() ?: 100 randomNumber = (minVal..maxVal).random() }) { Text("Generate", fontSize = 18.sp) } } }
```

App 12: BMI Calculator

Calculate Body Mass Index

```
@Composable fun BMICalculator() { var height by remember { mutableStateOf("") } var weight by remember { mutableStateOf("") } var bmi by remember { mutableStateOf<Double?>(null) } Column( modifier = Modifier .fillMaxSize() .padding(16.dp), verticalArrangement = Arrangement.Center ) { Text("BMI Calculator", fontSize = 24.sp, fontWeight = FontWeight.Bold) Spacer(modifier = Modifier.height(16.dp)) OutlinedTextField( value = height, onValueChange = { height = it }, label = { Text("Height (cm)") }, modifier = Modifier.fillMaxWidth() ) Spacer(modifier = Modifier.height(8.dp)) OutlinedTextField( value = weight, onValueChange = { weight = it }, label = { Text("Weight (kg)") }, modifier = Modifier.fillMaxWidth() ) Spacer(modifier = Modifier.height(16.dp)) Button( onClick = { val h = height.toDoubleOrNull()?.div(100) val w = weight.toDoubleOrNull() if (h != null && w != null && h > 0) { bmi = w / (h * h) } }, modifier = Modifier.fillMaxWidth() ) { Text("Calculate BMI") } bmi?.let { Spacer(modifier = Modifier.height(16.dp)) Text( text = "BMI: %.2f".format(it), fontSize = 32.sp, fontWeight = FontWeight.Bold ) Text( text = when { it < 18.5 -> "Underweight" it < 25 -> "Normal" it < 30 -> "Overweight" else -> "Obese" }, fontSize = 20.sp ) } } }
```

App 13: Tip Calculator

Calculate tip and total amount

```
@Composable fun TipCalculator() { var billAmount by remember { mutableStateOf("") } var tipPercent by remember { mutableStateOf(15f) } val tip = billAmount.toDoubleOrNull()?.let { it * tipPercent / 100 }?: 0.0 val total = billAmount.toDoubleOrNull()?.let { it + tip }?: 0.0 Column( modifier = Modifier .fillMaxSize() .padding(16.dp) ) { Text("Tip Calculator", fontSize = 24.sp, fontWeight = FontWeight.Bold) Spacer(modifier = Modifier.height(16.dp)) OutlinedTextField( value = billAmount, onValueChange = { billAmount = it }, label = { Text("Bill Amount") }, modifier = Modifier.fillMaxWidth() ) Spacer(modifier = Modifier.height(16.dp)) Text("Tip: ${tipPercent.toInt()}%") Slider( value = tipPercent, onValueChange = { tipPercent = it }, valueRange = 0f..30f, steps = 29 ) Spacer(modifier = Modifier.height(16.dp)) Card( modifier = Modifier.fillMaxWidth(), colors = CardDefaults.cardColors(containerColor = Color.LightGray) ) { Column(modifier = Modifier.padding(16.dp)) { Text("Tip: %.2f".format(tip), fontSize = 20.sp) Text("Total: %.2f".format(total), fontSize = 24.sp, fontWeight = FontWeight.Bold) } } }
```

App 14: Password Generator

Generate random secure passwords

```
@Composable fun PasswordGenerator() { var password by remember { mutableStateOf("") } var length by remember { mutableStateOf(12) } var includeNumbers by remember { mutableStateOf(true) } var includeSymbols by remember { mutableStateOf(true) } fun generatePassword() { val chars = buildString { append("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ") if (includeNumbers) append("0123456789") if (includeSymbols) append("!@#$%^&*()") } password = (1..length).map { chars.random() }.joinToString("") } Column( modifier = Modifier .fillMaxSize() .padding(16.dp) ) { Text("Password Generator", fontSize = 24.sp, fontWeight = FontWeight.Bold) Spacer(modifier = Modifier.height(16.dp)) Text("Length: $length") Slider( value = length.toFloat(), onValueChange = { length = it.toInt() }, valueRange = 4f..32f ) Row(verticalAlignment = Alignment.CenterVertically) { Text("Include Numbers") } Row(verticalAlignment = Alignment.CenterVertically) { Checkbox(checked = includeNumbers, onCheckedChange = { includeNumbers = it }) Text("Include Symbols") } Spacer(modifier = Modifier.height(16.dp)) Button(onClick = { generatePassword() }, modifier = Modifier.fillMaxWidth()) { Text("Generate Password") } if (password.isNotEmpty()) { Spacer(modifier = Modifier.height(16.dp)) SelectionContainer { Text(password, fontSize = 18.sp, fontWeight = FontWeight.Bold) } } }
```

App 15: Dice Roller App

Roll dice with animation

```
@Composable fun DiceRollerApp() { var diceValue by remember { mutableStateOf(1) } var isRolling by remember { mutableStateOf(false) } LaunchedEffect(isRolling) { if (isRolling) { repeat(10) { diceValue = (1..6).random() delay(100) } isRolling = false } } Column( modifier = Modifier.fillMaxSize(), horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.Center ) { Text("Dice Roller", fontSize = 24.sp, fontWeight = FontWeight.Bold) Spacer(modifier = Modifier.height(32.dp)) Card( modifier = Modifier.size(120.dp), colors = CardDefaults.cardColors(containerColor = Color.Red) ) { Box( modifier = Modifier.fillMaxSize(), contentAlignment = Alignment.Center ) { Text( text = diceValue.toString(), fontSize = 72.sp, fontWeight = FontWeight.Bold, color = Color.White ) } } Spacer(modifier = Modifier.height(32.dp)) Button( onClick = { isRolling = true }, enabled = !isRolling ) { Text("Roll Dice", fontSize = 18.sp) } }
```